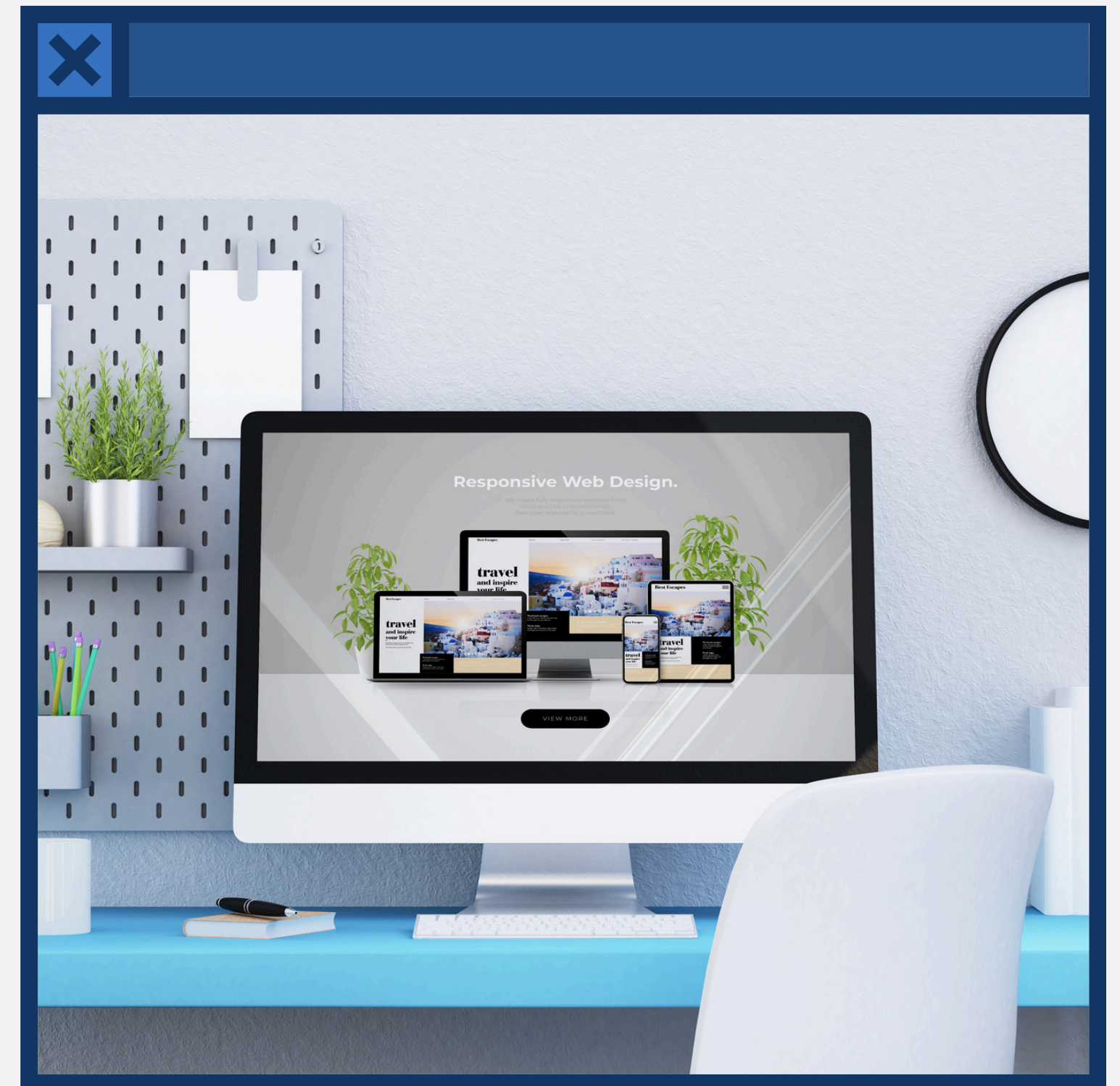


CSS Layouting & positioning

Muhammad Irsyad
13182420022



Bagian I: Fondasi Layouting

Apa itu CSS?

01 Cascading Style Sheets

Bahasa untuk mendesain presentasi dokumen web (HTML).

02 Separation of Concerns

Memisahkan struktur konten dari desain visual.

03 Interaktivitas

Memberikan pengalaman pengguna yang dinamis melalui transisi dan animasi.

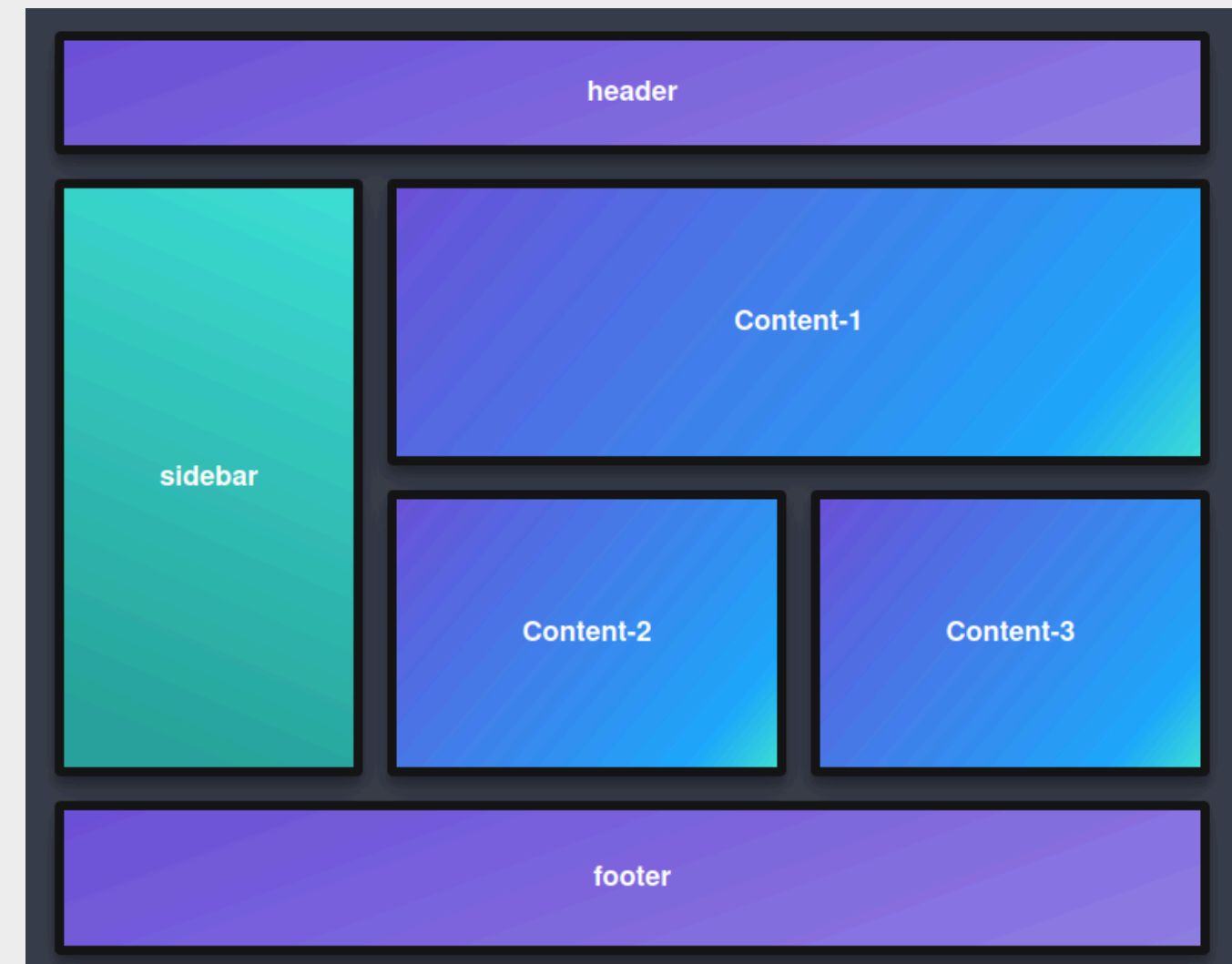
04 Responsivitas

Kemampuan beradaptasi di berbagai ukuran layar.

MENGAPA LAYOUT PENTING?

Layout bukan sekadar estetika. Tata letak yang baik menentukan kemudahan navigasi, hierarki informasi, dan kecepatan pemahaman pengguna terhadap konten Anda.

Studi menunjukkan pengguna hanya membutuhkan 0.05 detik untuk membentuk opini tentang sebuah website berdasarkan tampilan visualnya.





TIGA PILAR DESAIN MODERN



Box Model



Positioning



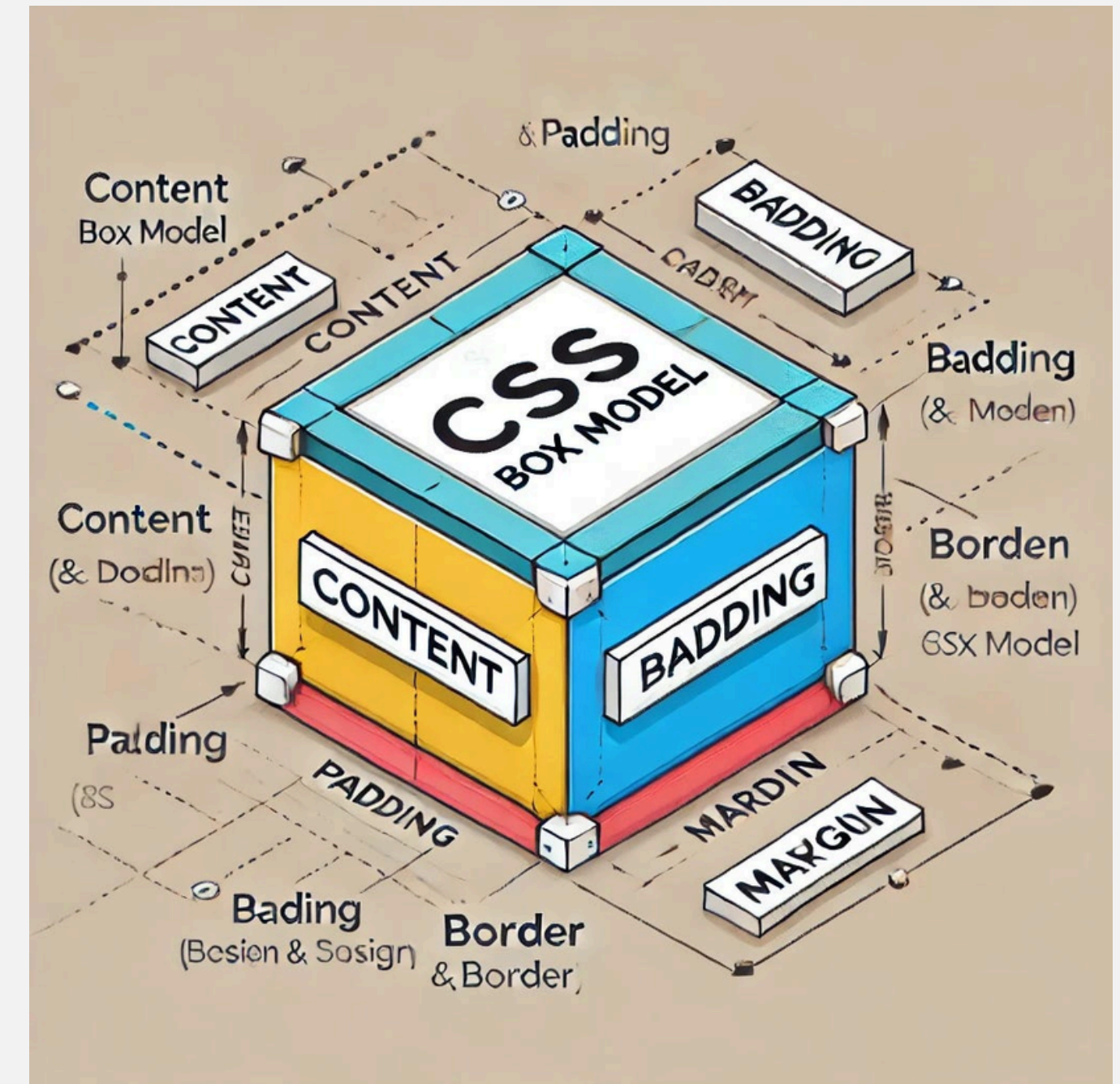
Layouting

Setiap elemen web adalah sebuah "kotak". Menguasai ketiga pilar ini adalah kunci menjadi Frontend Developer handal.

Bagian II: The Box Model

APA ITU BOX MODEL?

- Content: Area utama tempat teks atau gambar berada.
- Padding: Ruang transparan di antara konten dan border.
- Border: Garis yang mengelilingi padding dan konten.
- Margin: Ruang kosong di luar border untuk memisahkan antar elemen.





PADDING VS MARGIN

PADDING

Mempengaruhi ukuran internal kotak. Menambah ruang di dalam border. Berguna untuk menjauhkan teks dari tepi elemen.

Margin

Mempengaruhi ruang eksternal. Digunakan untuk mengatur jarak antar komponen yang berbeda di halaman web.

Auto Margin

Menggunakan margin: auto sering digunakan untuk memosisikan elemen block ke tengah secara horizontal di dalam parent-nya.

PROPERTI BOX-SIZING

Value	Deskripsi	Efek pada Lebar (Width)
content-box (default)	Padding dan Border ditambahkan ke lebar elemen.	Width + Padding + Border
border-box	Padding dan Border sudah termasuk dalam lebar elemen.	Tetap sesuai nilai Width
inherit	Mengikuti nilai box-sizing dari elemen induknya.	Variabel

Bagian III: Display Property



BLOCK, INLINE, & INLINE-BLOCK

Block

Memulai baris baru.
Mengambil seluruh lebar yang tersedia.
Contoh: `<div>`, `<p>`, `<h1>`.

Inline

Tidak memulai baris baru. Lebarnya hanya sebesar konten.
Contoh: `<span>`, `<a>`, `<strong>`.

Inline-Block

Tetap sejajar (inline) tapi bisa menerima properti Box Model (width, height, padding, margin).

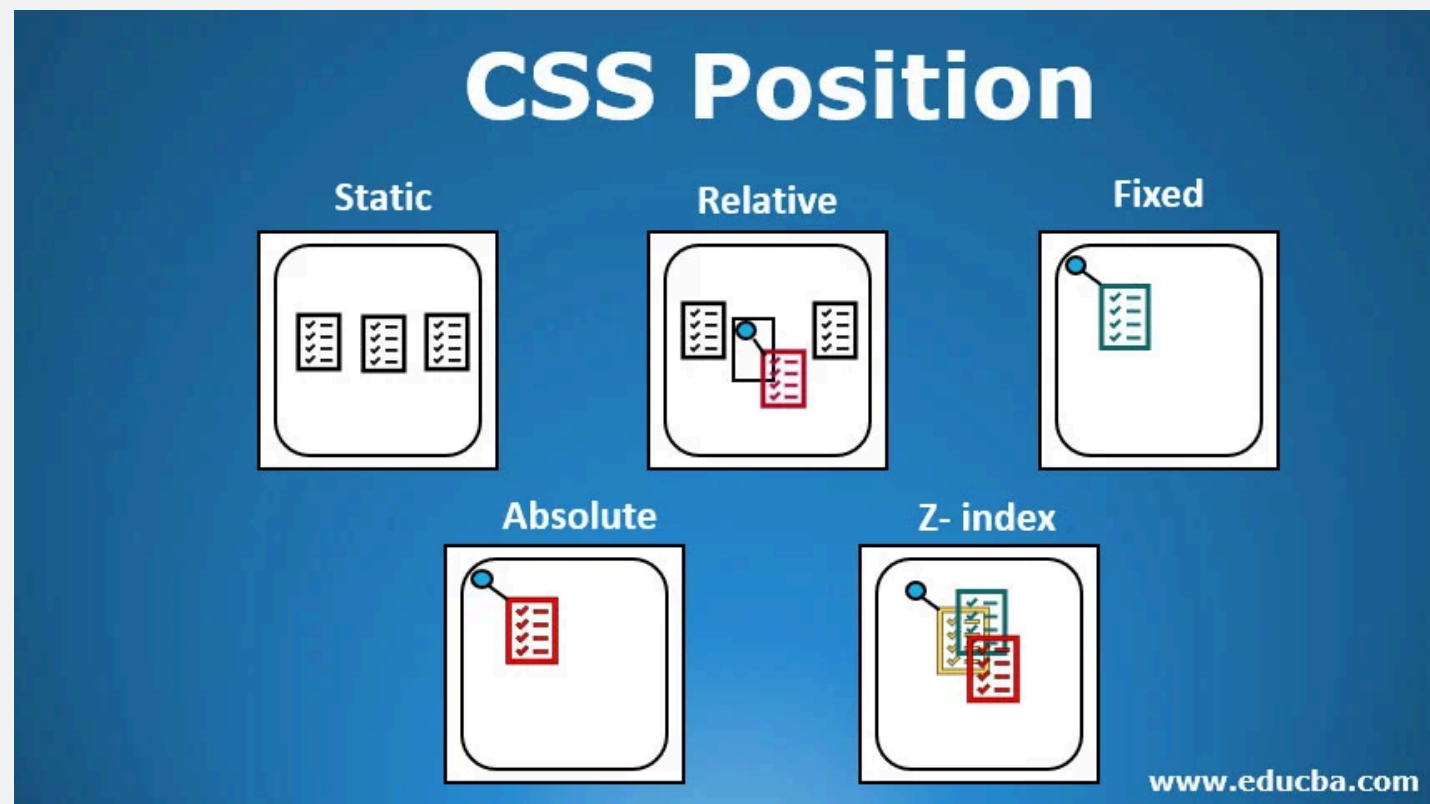


MENYEMBUNYIKAN ELEMEN

- `display: none`: Elemen dihapus dari layout. Ruang yang ditinggalkan akan diisi elemen lain.
- `visibility: hidden`: Elemen tidak terlihat, tetapi ruang yang ditempati tetap dipertahankan di layout.
- `opacity: 0`: Elemen transparan penuh, tetap bisa berinteraksi (clickable) dan mempertahankan ruang.

Bagian IV: Positioning

STATIC & RELATIVE POSITIONING



- Static: Posisi default. Elemen mengikuti aliran normal dokumen.
- Relative: Elemen diposisikan relatif terhadap posisi aslinya.
- Menggunakan properti top, right, bottom, left untuk pergeseran tanpa mengganggu elemen lain.
- Sering digunakan sebagai "jangkar" untuk elemen anak berposisi absolute.

ABSOLUTE POSITIONING

Elemen "melayang" keluar dari aliran normal dokumen. Ia akan mencari elemen induk (ancestor) terdekat yang memiliki posisi selain static.

Sangat ideal untuk membuat lencana (badges), ikon overlay, atau dropdown menu yang harus menumpuk di atas konten lain.





FIXED & STICKY POSITIONING

Fixed

Relatif terhadap viewport (layar). Tetap di tempat yang sama meskipun halaman di-scroll. Umum untuk Navbar.

Sticky

Campuran relative dan fixed. Elemen mengikuti scroll hingga mencapai batas tertentu, lalu "menempel".

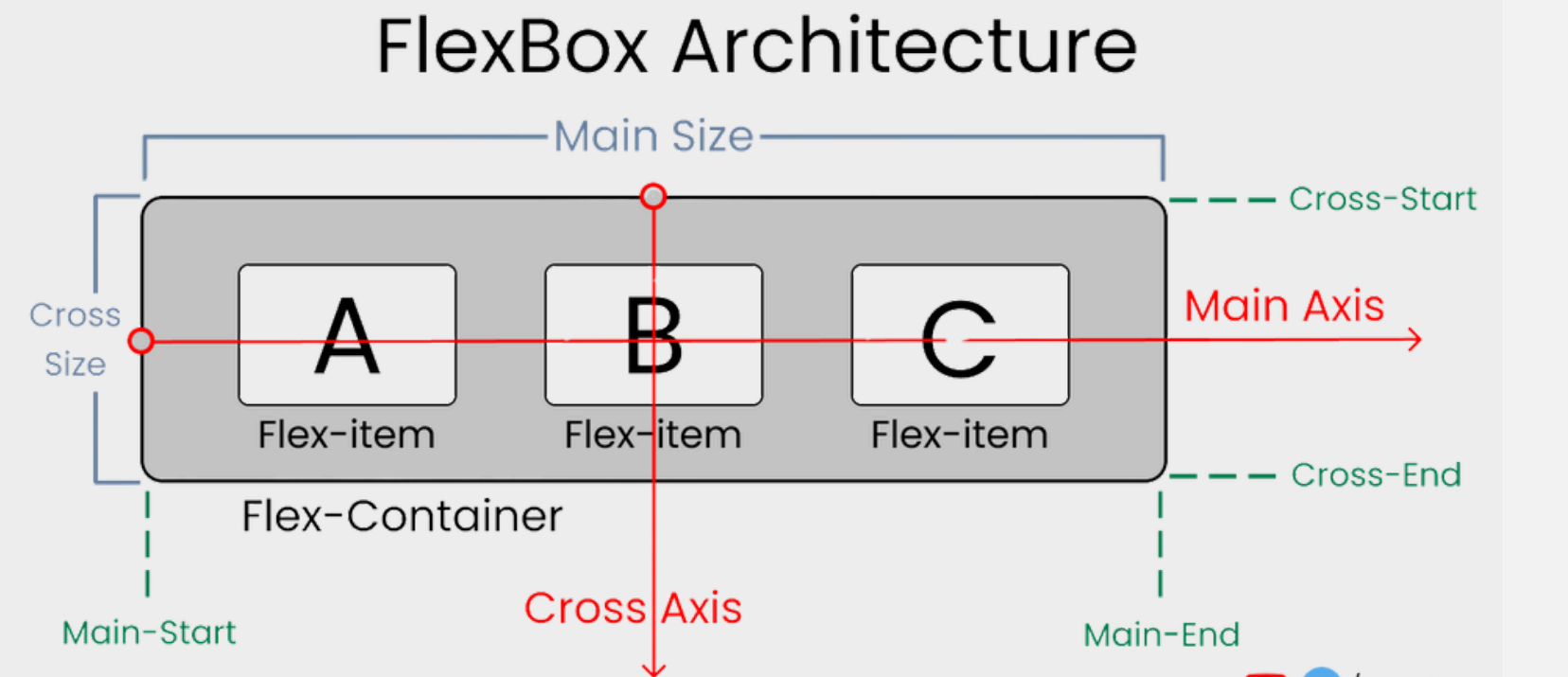
Z-Index

Mengatur urutan tumpukan elemen di sumbu Z. Nilai lebih tinggi akan berada di atas nilai rendah.

Bagian V: Flexbox Layout

KONSEP DASAR FLEXBOX

- Flex Container: Elemen induk (display: flex).
- Main Axis: Sumbu utama tempat item dijejerkan.
- Cross Axis: Sumbu yang tegak lurus dengan Main Axis.
- Mampu mengubah urutan, ukuran, dan perataan secara dinamis.





ARAH & PEMBUNGKUSAN

flex-direction

Row (default),
Column, Row-reverse,
Column-reverse.
Menentukan arah
sumbu utama.

flex-wrap

Menentukan apakah
item harus dipaksa
dalam satu baris atau
boleh pindah ke baris
baru jika ruang tidak
cukup.

gap

Cara modern
memberikan jarak
antar item di dalam
flex container tanpa
menggunakan
margin.

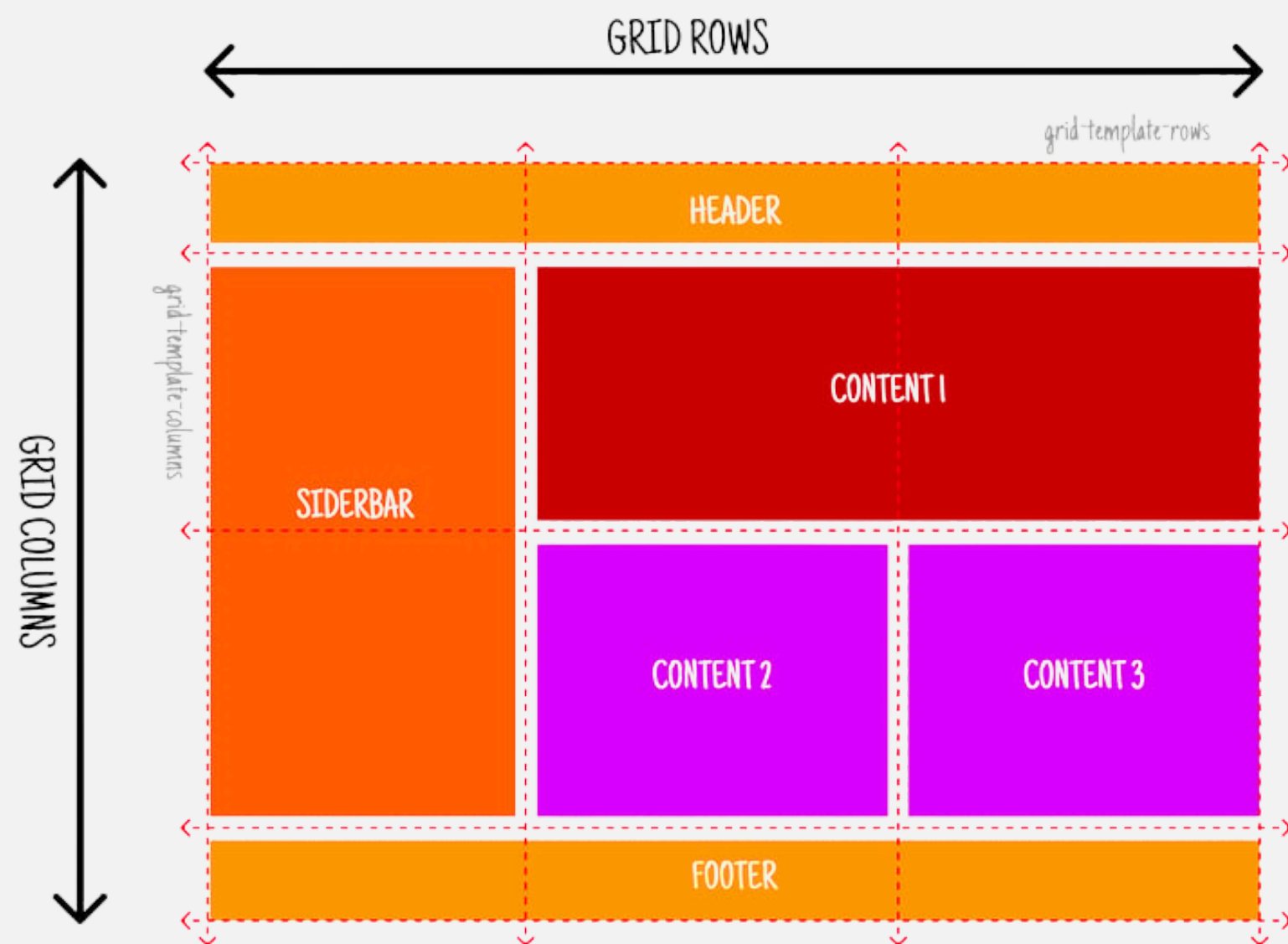


ALIGNMENT PROPERTIES

- justify-content: Perataan di sumbu utama (center, space-between, space-around).
- align-items: Perataan di sumbu silang untuk semua item (stretch, center, flex-start).
- align-self: Mengatur perataan spesifik untuk satu item tertentu, mengabaikan align-items kontainer.
- flex-grow / flex-shrink: Mengontrol bagaimana item membesar atau mengecil untuk mengisi ruang.

Bagian VI: CSS Grid

2D GRID SYSTEM



- Display Grid: Mengaktifkan konteks grid (Baris & Kolom).
- fr Unit: Fractional unit untuk membagi ruang secara proporsional.
- Grid Template Areas: Menamai area layout untuk kemudahan struktur.
- Ideal untuk dashboard dan struktur halaman utama yang kompleks.



GRID TEMPLATE & GAPS

`grid-template-columns`

Menentukan jumlah dan lebar kolom.
Contoh: `repeat(3, 1fr)` menciptakan 3 kolom sama lebar.

`grid-template-rows`

Menentukan jumlah dan tinggi baris. Bisa menggunakan `min-content` atau `max-content`.

`gap (row & column)`

Mengatur jarak presisi antar sel tanpa mempengaruhi ukuran total kotak (box model).



ALASAN MEMILIH CSS GRID

- Kontrol Dua Dimensi (Baris & Kolom)
- Kesederhanaan Kode (Sedikit elemen pembungkus)
- Kemudahan Re-order Elemen untuk Mobile

GRID VS FLEXBOX

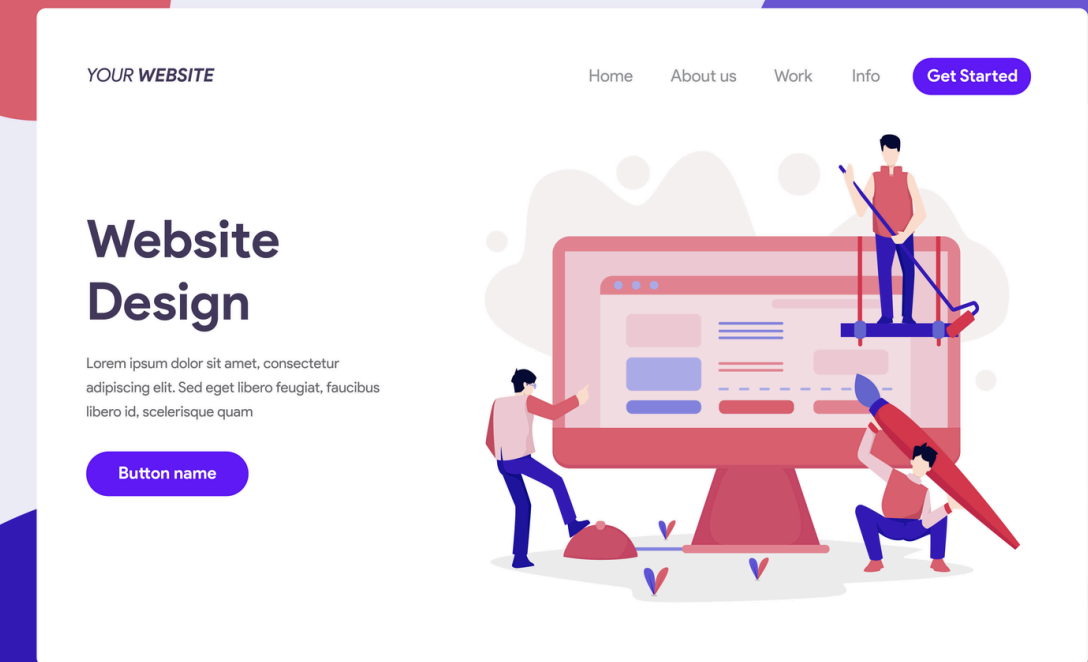
Fitur	Flexbox	CSS Grid
Dimensi	1D (Horizontal atau Vertikal)	2D (Horizontal & Vertikal)
Fokus	Content-first (Isi elemen)	Layout-first (Struktur luar)
Penggunaan	Navbar, Card List, Tombol	Dashboard, Layout Artikel, Galeri

Bagian VII: Responsivitas

MEDIA QUERIES

Memungkinkan penerapan gaya CSS yang berbeda berdasarkan karakteristik perangkat, seperti lebar layar.

Pendekatan Mobile First: Menulis kode dasar untuk mobile, lalu menambah fitur untuk layar besar melalui media queries.





BREAKPOINT STANDAR

640

Mobile (px)

768

Tablet (px)

1024

Desktop (px)

1280

Large (px)



SATUAN CSS MODERN

rem / em

Relatif terhadap ukuran font. rem (Root) vs em (Parent). Bagus untuk aksesibilitas.

vw / vh

Viewport Width / Height. 100vw = 100% lebar layar. Berguna untuk hero section.

Clamp()

Fungsi modern untuk menentukan rentang nilai (min, ideal, max). Berguna untuk fluid typography.



MASA DEPAN CSS

- Container Queries: Responsivitas berdasarkan ukuran kontainer induk, bukan layar.
- :has() Selector: "Parent selector" yang paling dinanti, memungkinkan styling parent berdasarkan anaknya.
- CSS Nesting: Menulis CSS berjenjang secara native tanpa perlu SASS/LESS.
- Subgrid: Memungkinkan sel anak mengikuti garis grid dari elemen induk.



THANK YOU

